

Correct and unsafe

Adrian Tame · 2026-08-19 ·

Most regex benchmarks ask one question: does the pattern pass the tests?

That is a reasonable question and it is not the one that decides whether you can deploy the answer. A regular expression can satisfy every example it was given and still hang your server. It can satisfy every example and still describe a different language than the one you asked for. Neither failure shows up in a pass rate.

So we measured all three. Eleven models, 450 tasks from [Re\(gEx|DoS\)Eval](#), three samples each, 14,850 calls, run on 2026-08-12. The full table is at [/benchmarks/](#).

The result

Every model passes roughly **40%** of tasks. Every model produces something shippable on roughly **20%**.

That gap is the finding, and it is remarkably stable. Across a 100x price range, the eleven models land within eight points of each other on the metric that matters: `kimik3` leads at 24.8% usable@3, `gemini-3.1-flash-lite` trails at 17.1%, and everything else is in between.

Three numbers tell the story:

- `pass@3` is what other benchmarks report. Did the pattern satisfy the examples.
- `vulnerable@3` is how many answers can be made to hang, by catastrophic backtracking.
- `usable@3` is what survives once you remove the vulnerable patterns *and* the ones provably describing a different language than the reference.

It passed every test and it can hang your server

Asked for a pattern that *"tests the validity of a domain or hostname"*, `claude-opus-5` answered:

```
^([a-zA-Z0-9]([a-zA-Z0-9\-\]{0,61}[a-zA-Z0-9])?\.\.)+(com|org|net|mil|edu)$
```

That is 100% correct on every example it was given. It is also exponentially vulnerable: a quantifier wraps a quantified group, so a failing suffix drives exponential backtracking.

This is not a contrived pattern. It looks like production code, it reads like something a careful engineer wrote, and putting it on a signup form gives you a denial-of-service bug.

135 such patterns appeared across the run: correct, and unsafe.

It passed every test and it is still wrong

Asked to match *"5 numeric digits, such as a zip code"*, the model answered `\b\d{5}\b` where the reference is `^\d{5}$`.

Both pass the tests. They are not the same pattern, and there is a string that proves it: `"\n000000"`. The model's version matches a zip code sitting on the second line of a multi-line string. The reference does not.

Whether that difference matters depends on your input, which is exactly why the benchmark reports it rather than silently calling one of them correct. **806 answers** passed their tests while describing a different language than the reference.

The most interesting failure is ours

Some of what we score as wrong is the model being right and the human-written reference being wrong.

Asked for *"a very simple ISBN validation expression, it just checks for a 10 digit number"*, the model wrote `^\d{9}[\dX]$`. The reference says `^\d{9}[\d\X]$`. They differ on the input `000000000|`.

The reference's character class contains a literal pipe. Someone wrote `[\d|X]` meaning "a digit or X" and accidentally allowed `|` as well. The model is correct. The gold answer has a typo. We score the model down for it.

Another: for *"positive integer value"* the model wrote `^[1-9][0-9]*$` and the reference `^\d+$`, differing on `0`. Zero is not a positive integer. The model is arguably right there too.

We have not audited how often this happens, so **treat the semantic-difference number as a lower bound on model correctness, not a verdict on it.** Publishing this is cheaper than having someone else find it.

Price buys very little here

`deepseek-v4-flash-0731` costs \$0.000026 per task and scores 19.8%. `claude-opus-5` costs \$0.002514 per task, **97x more**, and scores 23.0%.

Three points for two orders of magnitude. Whatever the frontier models are better at, writing a regular expression you could ship is not obviously it.

Failures are results too

54 of 14,850 calls failed, 0.36%. They are in the table rather than dropped, because a model that declines to answer has not earned the same denominator as one that answers badly.

`claude-opus-5` was refused by a content filter on 29 calls, on benign prompts. That is a property of the deployment, not the model, and it belongs in the result for the same reason.

Check it yourself

Every model response is committed. Scores are computed from those files and nothing else, so you can recheck the arithmetic without an API key, without spending anything, and without trusting us:

```
git clone https://github.com/foohills-labs/regexleaderboard
cd regexleaderboard
make setup      # installs the pinned scorer, downloads the corpus
make score RUN=sweep
```

`make check` does the same and fails if the recomputed numbers differ from the published ones. It runs in CI on every push, so the scoring path cannot silently drift.

The study repository holds the data, the method and the re-run command; this article is the narrative and links to it. Scoring is by `regexbench` 0.4.0. The corpus is `Re(gEx|DoS)Eval`, not redistributed; `make setup` fetches it from source.

Numbers: [/benchmarks/](#). Data, method and limitations: [regexleaderboard](#).